

Relatório final — De Spec a Código

Aluno: Lucas Santos **Disciplina:** Tópicos Avançados em Engenharia de Software 2 — PPgTI / IMD-UFRN **Repositório:** <https://github.com/lucassmsantoss/ia-dev-lab> **Pull Request:** <https://github.com/lucassmsantoss/ia-dev-lab/pull/2> (aberto, sem merge) **Data:** 02/09/2026

1. Funcionalidades escolhidas e por que eram bom caso para SDD

Escolhi duas funcionalidades no domínio de validação de documentos do projeto iniciado na Aula 2: **validação de CNPJ com suporte ao formato alfanumérico** e **validação em lote a partir de arquivo CSV**.

A primeira é o melhor caso de SDD que eu poderia ter encontrado, e por um motivo específico: a Receita Federal começou a emitir CNPJ alfanumérico em **31/07/2026**, há dois meses. As 12 primeiras posições passaram a aceitar letras, apenas os dígitos verificadores permanecem numéricos, e cada caractere vale seu código ASCII menos 48. Um modelo que “sabe validar CNPJ” sabe a versão anterior a julho — e escreveria, com total convicção, um validador que **rejeita como inválida a inscrição de qualquer empresa aberta depois dessa data**, aprovado por testes que ele mesmo geraria. Nenhum refinamento de prompt corrige isso, porque o erro não está na formulação do pedido: está na premissa. Só escrever a regra antes, conferindo a fonte, evita.

A segunda entrou por ser de outra natureza: tem I/O, dois consumidores com necessidades diferentes (biblioteca e linha de comando) e casos de borda que vêm do mundo real — CSV do Excel com BOM, separador `;`, coluna ausente. Serve para testar se o SDD ajuda também onde a regra não é externa.

2. Abordagens usadas e como se comportaram

Usei **OpenSpec 1.11.0** para o CNPJ, gerando sete artefatos, e **Markdown manual** para o lote, em um arquivo. O código final ficou equivalente — 50 e 28 testes, ambos atendendo integralmente à respectiva especificação. A disciplina importou mais que a ferramenta.

Houve, porém, uma diferença que não é de gosto. O OpenSpec separa a proposta do plano em arquivos distintos, e foi isso que permitiu **comparar um contra o outro**: das nove tarefas propostas pelo agente para o CNPJ, duas violavam não-objetivos declarados — formatar com máscara e gerar CNPJs para os testes. A segunda era o pior erro do plano, porque testes alimentados por um gerador só provam que gerador e validador concordam, ainda que ambos errem a regra. Removi as duas, fundi três que eram a mesma tarefa e acrescentei a que faltava: conferir a norma antes de escrever código.

Na spec manual isso não aconteceu — e não porque o plano fosse melhor. Um documento único não tem seção de não-objetivos destacada, então **não havia contra o que comparar**. O valor não estava na ferramenta, estava na separação física entre “o que decidimos não fazer” e “o que vamos fazer”.

3. Dificuldade real enfrentada

A dificuldade foi conceitual: **descobrir o que a especificação não consegue proteger**.

Entrei na atividade tratando a spec como a rede de segurança principal, e ela falhou duas vezes, de formas diferentes. Primeiro, o critério de aceite CA2 do lote afirmava que uma linha em branco seria reportada como “campo vazio”. O teste falhou — o módulo `csv` da biblioteca padrão descarta linhas em branco antes de entregá-las. Aqui a **especificação estava errada e o código estava certo**, o inverso do que o método promete. Corrigi o critério e acrescentei o CA2b.

Depois, escrevi os testes do lote usando `Path.write_text(..., newline=...)`. A verificação passou, e ao rodar `pytest` na máquina do projeto **13 dos 28 casos quebraram**: esse parâmetro só existe a partir do Python 3.10, e o ambiente é 3.9.12. A regra estava escrita no `CLAUDE.md` do próprio repositório. Nem a spec nem a revisão de diff pegariam — a spec descreve comportamento, não compatibilidade de interpretador, e a lógica estava correta. O que falhou foi o veredito “verificado” ter sido emitido por um ambiente que não era o de destino.

O aprendizado é que especificação e teste não são redundantes, e sim complementares: a primeira verifica se estamos construindo a coisa certa, o segundo verifica se a construímos direito. Neste trabalho aconteceu uma falha de cada tipo — e nenhuma das duas teria sido pega pelo instrumento que cobre a outra.

Isso não diminui o SDD; delimita onde ele paga. Ele paga quando a regra vem de fora e não pode ser adivinhada, e quando o escopo precisa ser defendido contra tarefas que parecem úteis. Não paga contra o comportamento das bibliotecas nem contra o ambiente de execução — e tratá-lo como se pagasse foi a parte difícil de desaprender.

Checklist de entregáveis

- ☒ `docs/escopo.md` com as funcionalidades escolhidas e a justificativa
- ☒ Especificação completa: requisitos, critérios de aceite com caso de borda próprio e plano de tarefas revisado — OpenSpec em `openspec/changes/add-validacao-cnpj/` e Markdown em `docs/spec-lote-csv.md`
- ☒ Registro das ferramentas e abordagens usadas, e por quê, em `docs/comparacao-abordagens-sdd.md`
- ☒ Repositório no GitHub com histórico de commits granular e Pull Request aberto
- ☒ Relatório final de uma página cobrindo os três pontos do roteiro

Registros adicionais: revisão do plano em `revisao-do-plano.md`, revisão dos diffs com três defeitos encontrados em `revisao-dos-diffs.md`, e o checkpoint humano em `checkpoint-humano.md`.

Evidência de execução: `python -m pytest -q` → **116 passed** (38 CPF, 50 CNPJ, 28 lote), em Anaconda com Python 3.9.12 e `pytest 7.1.1`.